

These expressions are virtually identical to those for $J_n(x)$ and $Y_n(x)$ in this region, except for the factor of $-2/\pi$ difference between $Y_n(x)$ and $K_n(x)$. In the region $x \gg n$, however, the modified functions have quite different behavior than the Bessel functions,

$$\begin{aligned} I_n(x) &\approx \frac{1}{\sqrt{2\pi x}} \exp(x) \\ K_n(x) &\approx \frac{\pi}{\sqrt{2\pi x}} \exp(-x) \end{aligned} \quad (6.5.15)$$

The modified functions evidently have exponential rather than sinusoidal behavior for large arguments (see Figure 6.5.2). Rational approximations analogous to those for the J and Y Bessel functions are efficient for computing I_0 , I_1 , K_0 , and K_1 . The corresponding routines are packaged as an object `Bessik`. The routines are similar to those in [1], although different in detail. (All the constants are again listed in a Webnote [3].)

```
bessel.h  struct Bessik {
    static const Doub i0p[14], i0q[5], i0pp[5], i0qq[6];
    static const Doub i1p[14], i1q[5], i1pp[5], i1qq[6];
    static const Doub k0pi[5], k0qi[3], k0p[5], k0q[3], k0pp[8], k0qq[8];
    static const Doub k1pi[5], k1qi[3], k1p[5], k1q[3], k1pp[8], k1qq[8];
    Doub y, z, ax, term;

    Doub i0(const Doub x) {
    Returns the modified Bessel function  $I_0(x)$  for any real x.
        if ((ax=abs(x)) < 15.0) {      Rational approximation.
            y = x*x;
            return poly(i0p,13,y)/poly(i0q,4,225.-y);
        } else {                      Rational approximation with  $e^x/\sqrt{x}$  factored out.
            z=1.0-15.0/ax;
            return exp(ax)*poly(i0pp,4,z)/(poly(i0qq,5,z)*sqrt(ax));
        }
    }

    Doub i1(const Doub x) {
    Returns the modified Bessel function  $I_1(x)$  for any real x.
        if ((ax=abs(x)) < 15.0) {      Rational approximation.
            y=x*x;
            return x*poly(i1p,13,y)/poly(i1q,4,225.-y);
        } else {                      Rational approximation with  $e^x/\sqrt{x}$  factored out.
            z=1.0-15.0/ax;
            Doub ans=exp(ax)*poly(i1pp,4,z)/(poly(i1qq,5,z)*sqrt(ax));
            return x > 0.0 ? ans : -ans;
        }
    }

    Doub k0(const Doub x) {
    Returns the modified Bessel function  $K_0(x)$  for positive real x.
        if (x <= 1.0) {                Use two rational approximations.
            z=x*x;
            term = poly(k0pi,4,z)*log(x)/poly(k0qi,2,1.-z);
            return poly(k0p,4,z)/poly(k0q,2,1.-z)-term;
        } else {                      Rational approximation with  $e^{-x}/\sqrt{x}$  factored out.
            z=1.0/x;
            return exp(-x)*poly(k0pp,7,z)/(poly(k0qq,7,z)*sqrt(x));
        }
    }
}
```

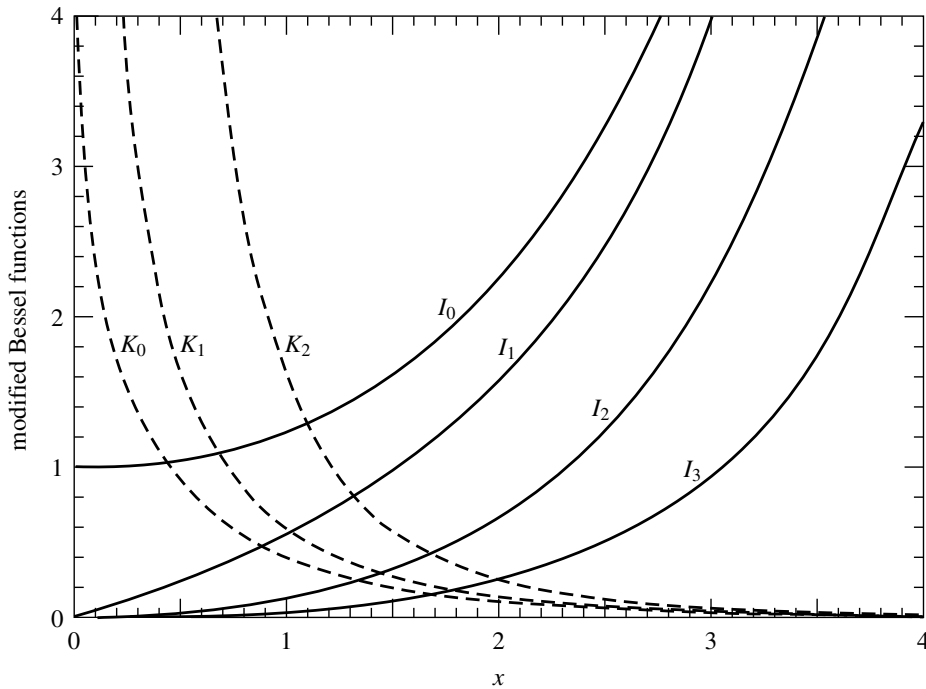


Figure 6.5.2. Modified Bessel functions $I_0(x)$ through $I_3(x)$, and $K_0(x)$ through $K_2(x)$.

```

Doub k1(const Doub x) {
Returns the modified Bessel function  $K_1(x)$  for positive real x.
    if (x <= 1.0) {
        Use two rational approximations.
        z=x*x;
        term = poly(k1pi,4,z)*log(x)/poly(k1qi,2,1.-z);
        return x*(poly(k1p,4,z)/poly(k1q,2,1.-z)+term)+1./x;
    } else {
        Rational approximation with  $e^{-x}/\sqrt{x}$  factored
        out.
        return exp(-x)*poly(k1pp,7,z)/(poly(k1qq,7,z)*sqrt(x));
    }
}

Doub in(const Int n, const Doub x);
Returns the modified Bessel function  $I_n(x)$  for any real x and  $n \geq 0$ .

Doub kn(const Int n, const Doub x);
Returns the modified Bessel function  $K_n(x)$  for positive x and  $n \geq 0$ .

inline Doub poly(const Doub *cof, const Int n, const Doub x) {
Common code: Evaluate a polynomial.
    Doub ans = cof[n];
    for (Int i=n-1;i>=0;i--) ans = ans*x+cof[i];
    return ans;
}
};

```

The recurrence relation for $I_n(x)$ and $K_n(x)$ is the same as that for $J_n(x)$ and $Y_n(x)$ provided that ix is substituted for x . This has the effect of changing a sign in the relation,

$$\begin{aligned}
 I_{n+1}(x) &= -\left(\frac{2n}{x}\right) I_n(x) + I_{n-1}(x) \\
 K_{n+1}(x) &= +\left(\frac{2n}{x}\right) K_n(x) + K_{n-1}(x)
 \end{aligned}
 \tag{6.5.16}$$

These relations are always *unstable* for upward recurrence. For K_n , itself growing, this presents no problem. The implementation is

```
bessel.h  Doub Bessik::kn(const Int n, const Doub x)
Returns the modified Bessel function  $K_n(x)$  for positive x and  $n \geq 0$ .
{
    Int j;
    Doub bk,bkm,bkp,tox;
    if (n==0) return k0(x);
    if (n==1) return k1(x);
    tox=2.0/x;
    bkm=k0(x);
    bk=k1(x);
    for (j=1;j<n;j++) {
        bkp=bkm+j*tox*bk;
        bkm=bk;
        bk=bkp;
    }
    return bk;
}
```

Upward recurrence for all x...

...and here it is.

For I_n , the strategy of downward recursion is required once again, and the starting point for the recursion may be chosen in the same manner as for the routine `Bessjy::jn`. The only fundamental difference is that the normalization formula for $I_n(x)$ has an alternating minus sign in successive terms, which again arises from the substitution of ix for x in the formula used previously for J_n :

$$1 = I_0(x) - 2I_2(x) + 2I_4(x) - 2I_6(x) + \dots \tag{6.5.17}$$

In fact, we prefer simply to normalize with a call to `i0`.

```
bessel.h  Doub Bessik::in(const Int n, const Doub x)
Returns the modified Bessel function  $I_n(x)$  for any real x and  $n \geq 0$ .
{
    const Doub ACC=200.0;
    const Int IEXP=numeric_limits<Doub>::max_exponent/2;
    Int j,k;
    Doub bi,bim,bip,dum,tox,ans;
    if (n==0) return i0(x);
    if (n==1) return i1(x);
    if (x*x <= 8.0*numeric_limits<Doub>::min()) return 0.0;
    else {
        tox=2.0/abs(x);
        bip=ans=0.0;
        bi=1.0;
        for (j=2*(n+Int(sqrt(ACC*n)));j>0;j--) {
            bim=bip+j*tox*bi;
            bip=bi;
            bi=bim;
            dum=frexp(bi,&k);
            if (k > IEXP) {
                ans=ldexp(ans,-IEXP);
                bi=ldexp(bi,-IEXP);
                bip=ldexp(bip,-IEXP);
            }
        }
    }
}
```

ACC determines accuracy.

Downward recurrence.

Renormalize to prevent overflows.